

Computer Architecture

Course No. 21472

Week #5

Arithmetic for Computers II

Bumsoo Kim

School of Computer Science & Engineering

E-mail : bumsoo@cau.ac.kr

Office : 310-430

Retrospect: Non-Leaf Procedure Example

- Example C code:

```
int fact (int n)
{
    if (n < 1) return 1;
    else return n * fact(n - 1);
}
```

- Argument n in $a0
- Result in $v0

```
fact:
    addi $sp, $sp, -8       # adjust stack for 2 items
    sw   $ra, 4($sp)        # save return address
    sw   $a0, 0($sp)        # save argument
    slti $t0, $a0, 1        # test for n < 1
    beq  $t0, $zero, L1
    addi $v0, $zero, 1      # if so, result is 1
    addi $sp, $sp, 8        #   pop 2 items from stack
    jr   $ra                #   and return
L1: addi $a0, $a0, -1       # else decrement n
    jal  fact               # recursive call
    lw   $a0, 0($sp)        # restore original n
    lw   $ra, 4($sp)        #   and return address
    addi $sp, $sp, 8        # pop 2 items from stack
    mul  $v0, $a0, $v0      # multiply to get result
    jr   $ra                # and return
```

$ra and $a0

$a0 < 1? If not -> L1

n * fact(n-1)

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Retrospect: Non-Leaf Procedure Example

- Write the C code and MIPS code for generating Fibonacci Sequence where

$$\text{fib}(n) = \begin{cases} n & \text{if } n < 2 \\ \text{fib}(n-1) + \text{fib}(n-2) & \text{if } n \geq 2 \end{cases}$$

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Arithmetic for Computers

Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow

Floating Point Real Numbers

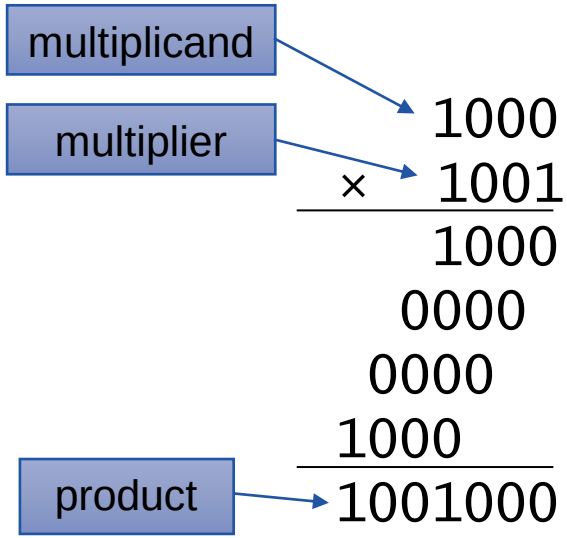
- Representation and Operation

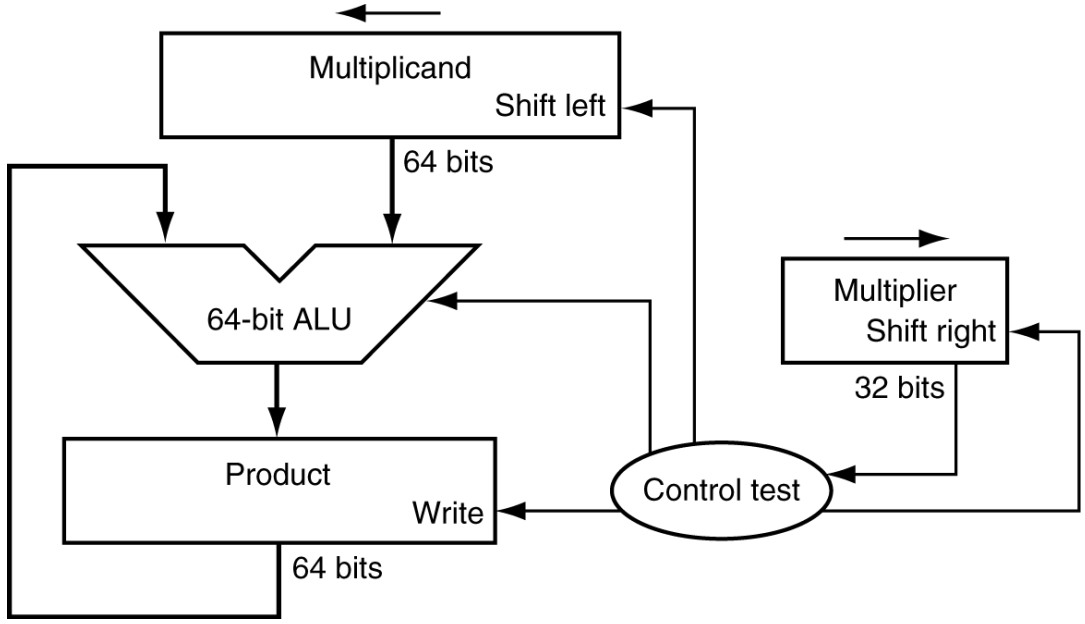
Precision

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow

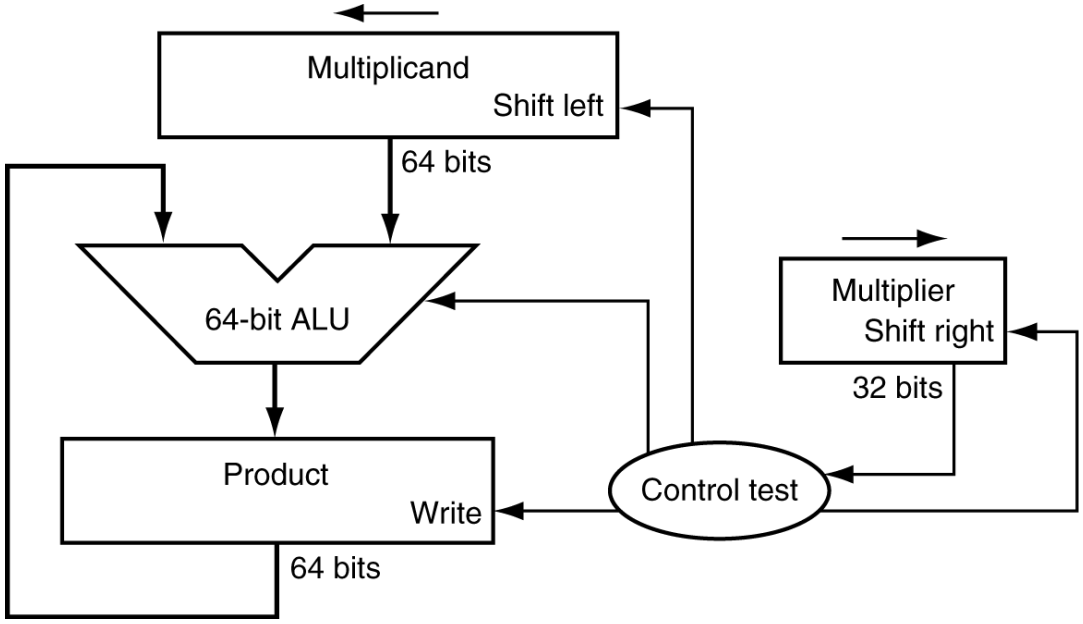


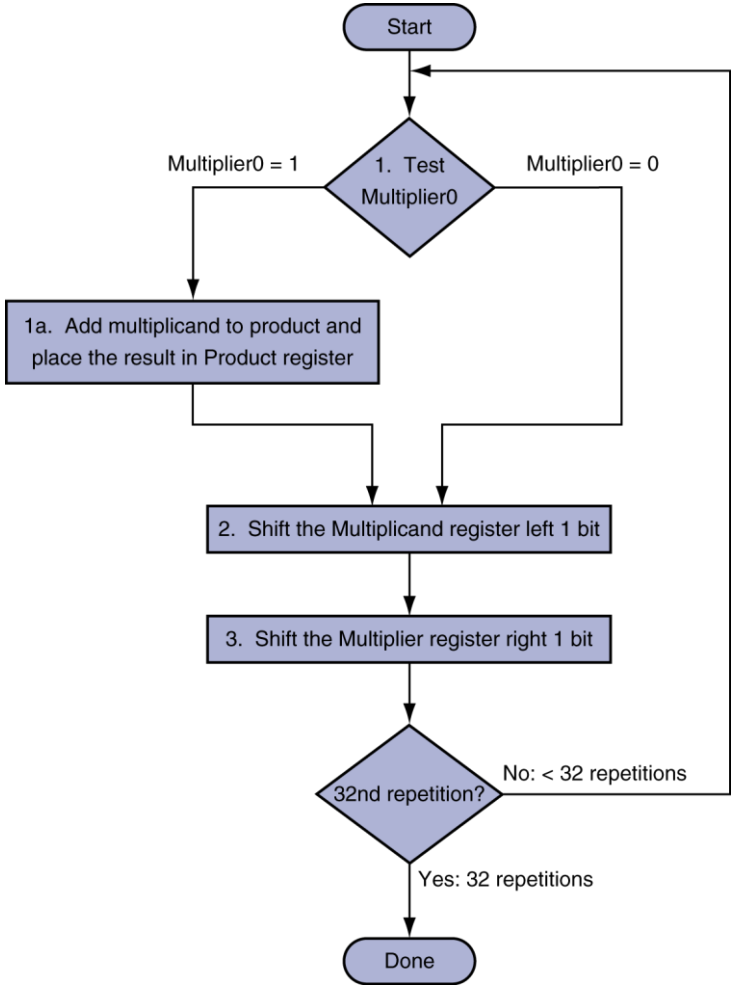


Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow

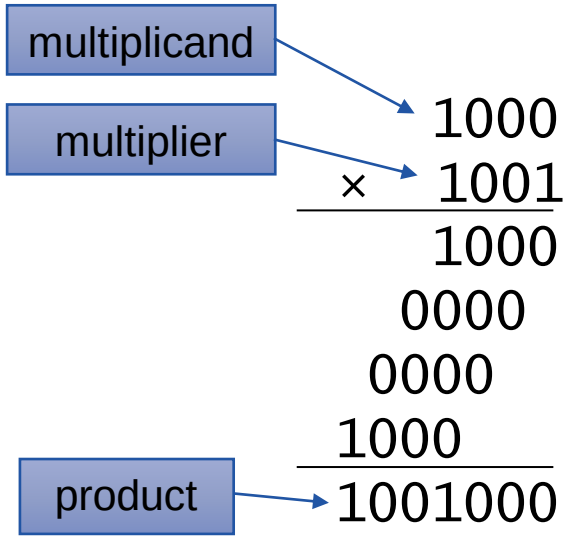


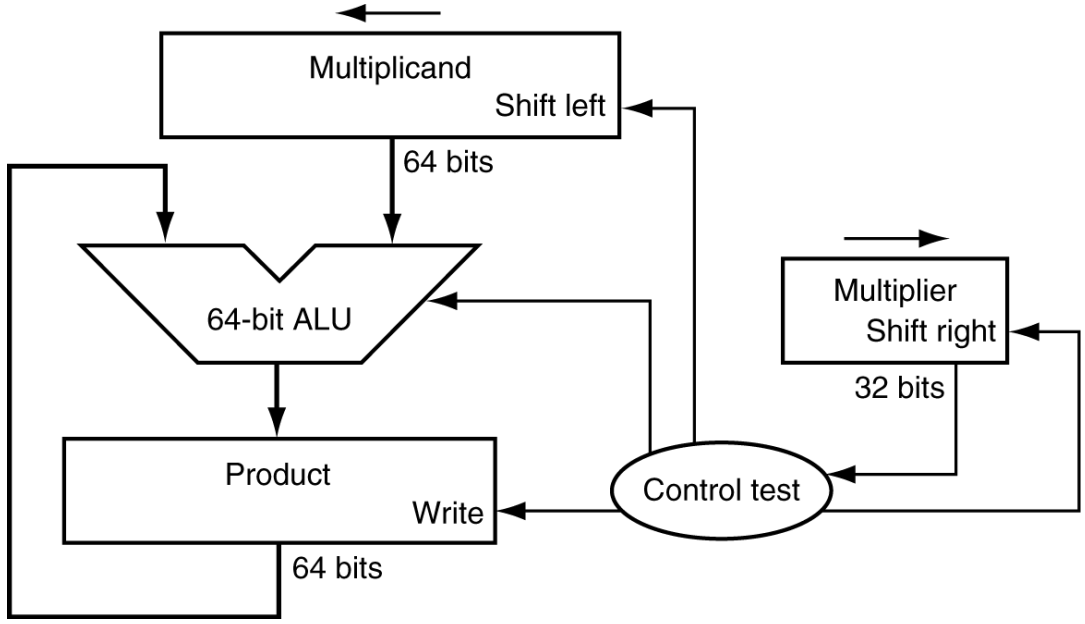


Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow

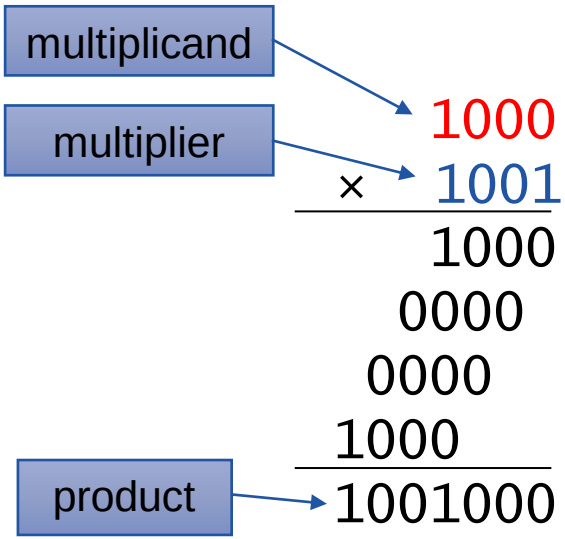


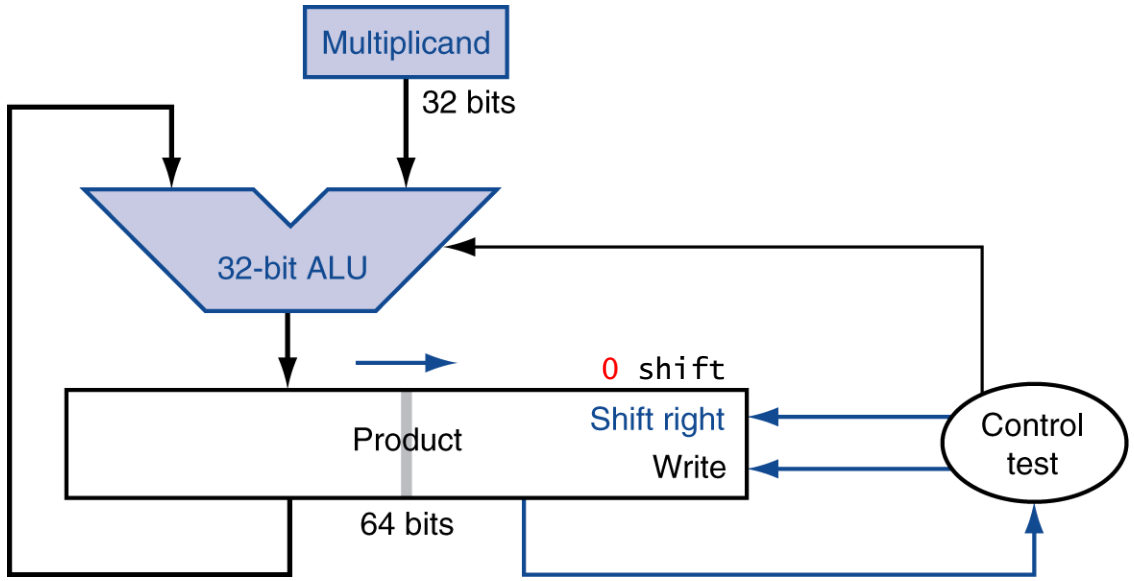


Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow





Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Arithmetic for Computers

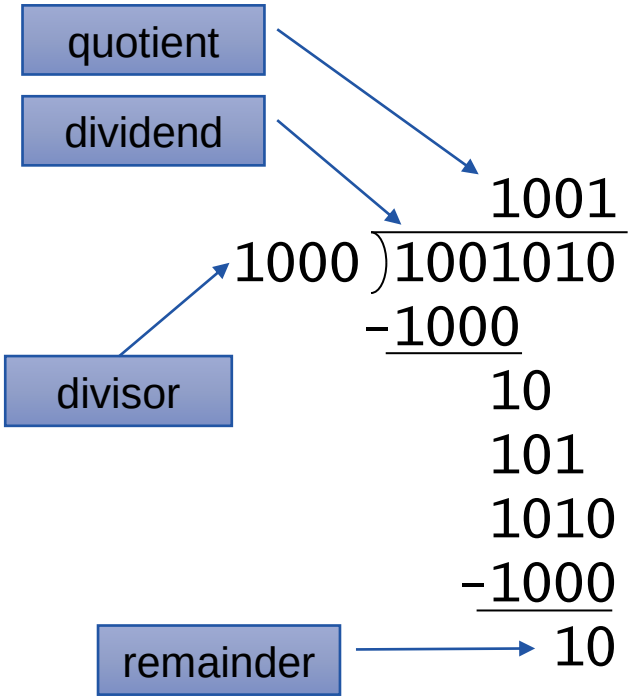
Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow

n-bit operands yield *n*-bit quotient and remainder

Dividend = Quotient × Divisor + Remainder

Always Check for 0 divisor!!



Long Division Approach

- If divisor ≤ dividend bits
  - 1 bit in quotient, subtract
- Otherwise
  - 0 bit in quotient, bring down next dividend bit

Restoring Division

- Do the subtract, and if remainder goes < 0, add divisor back

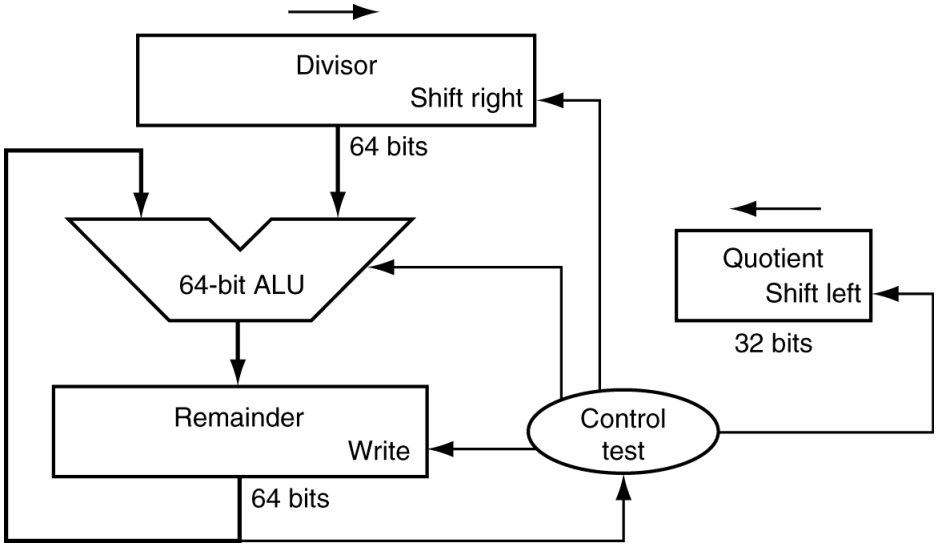
Signed Division

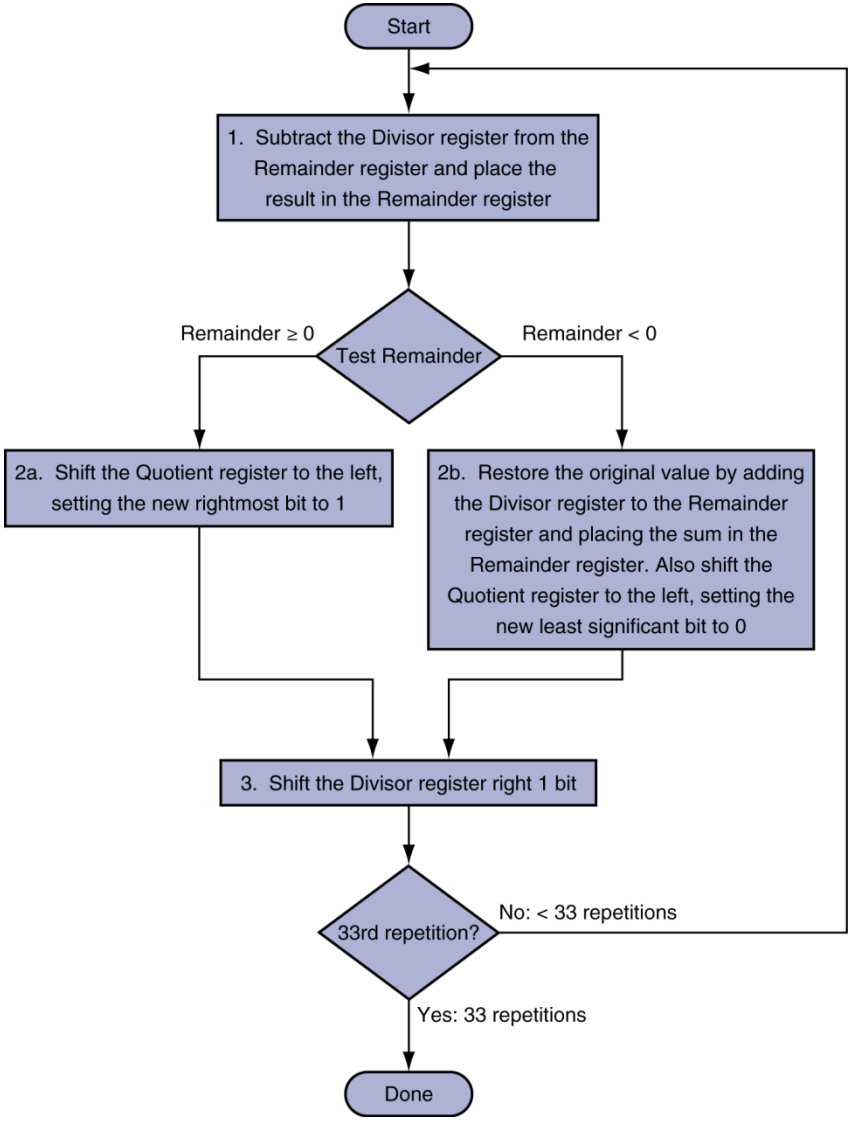
- Divide using absolute values
- Adjust sign of quotient and remainder as required
- Dividend and remainder must have the same signs

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow



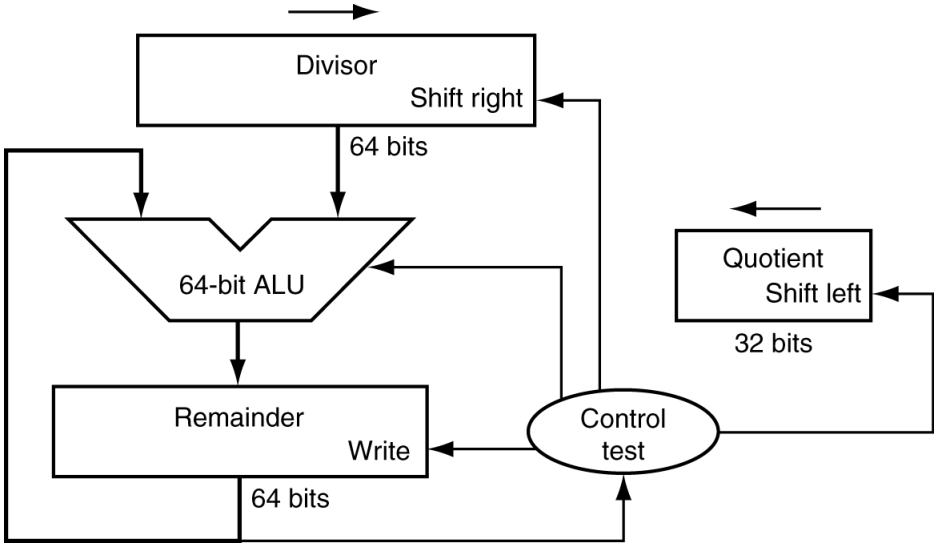


Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow

$$1000\overline{)01001010}$$

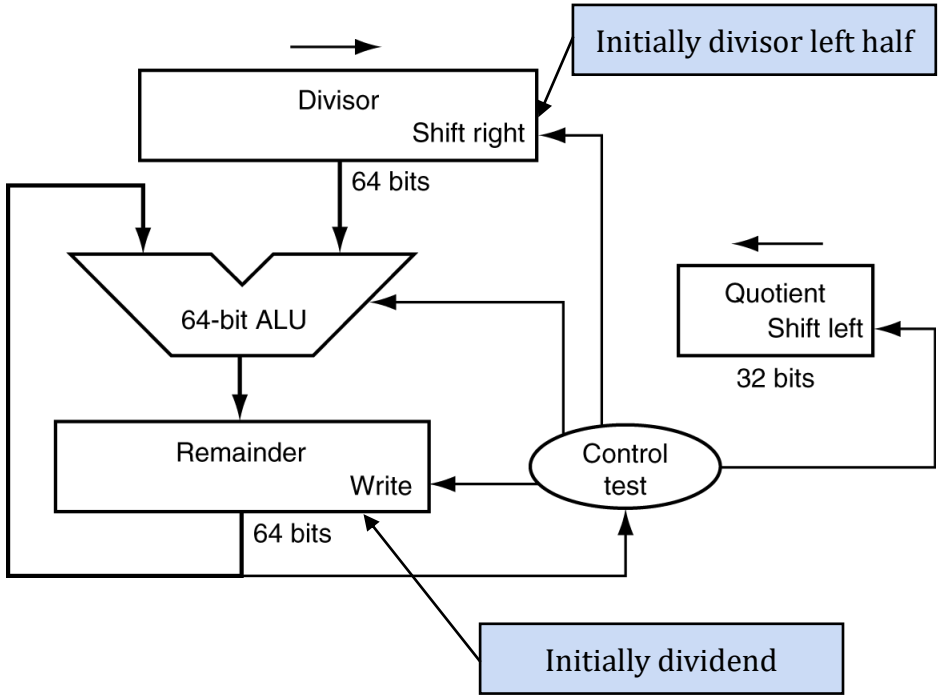


Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow

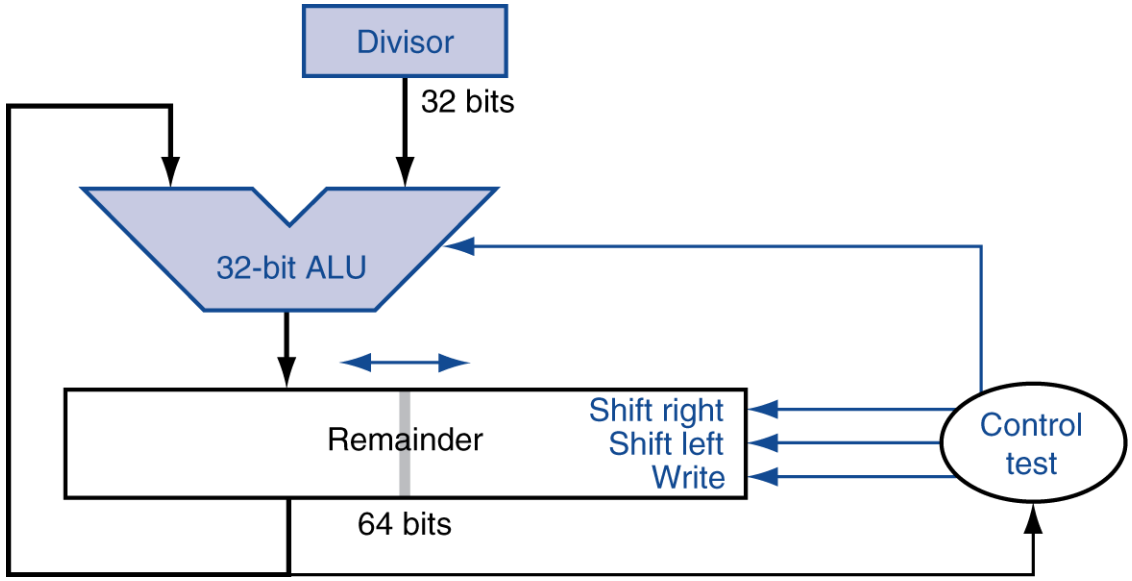
```
        01001
1000)01001010
   -1000
    1100 1010
   +1000
    0100 1010
    -1000
    0000 1010
     -1000
     111010 10
     +1000
     000010 10
      -1000
      111110 10
      +1000
      0000 1010
       -1000
       0000 0010
```



Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow

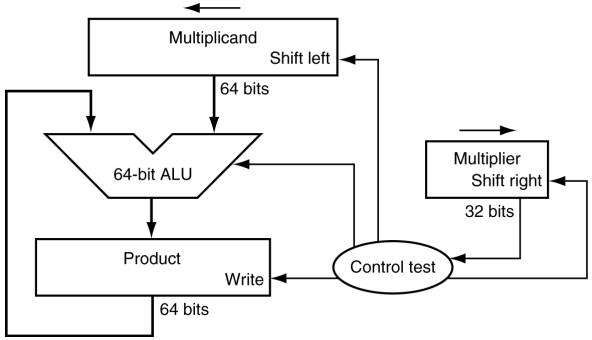


Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Example 1

Example [1]

Given the unit below, show the process how we calculate 1010 x 0010



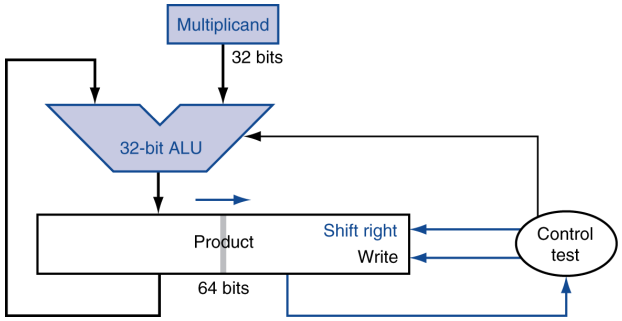
A.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Example 2

Example [2]

Given the unit below, show the process how we calculate 1010 x 1100



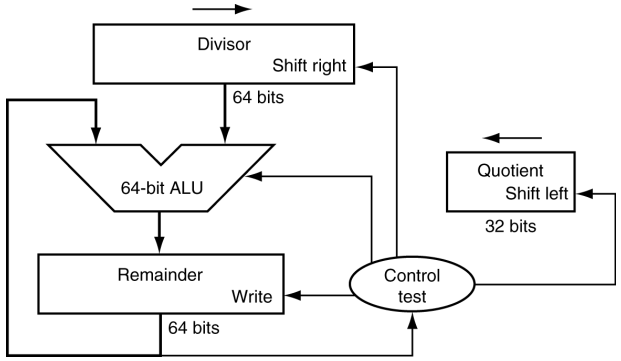
A.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Example 3

Example [3]

Given the unit below, show the process how we calculate 0001 0100 ÷ 0010



A.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Arithmetic for Computers

Operation on Integers

- Addition and Subtraction
- Multiplication and Division
- Dealing with Overflow

Floating Point Real Numbers

- Representation and Operation

Precision

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Floating Point Real Numbers

- Representation and Operation

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

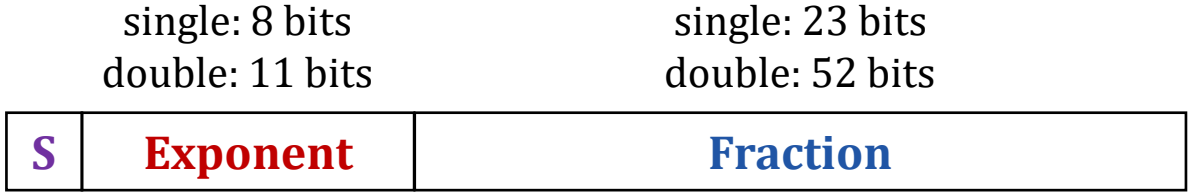
Floating Point Real Numbers

Floating Point Standard

- Defined by IEEE Std 754-1985
- Developed in response to divergence of representations
- Now almost universally adopted
- Two representations
  - Single precision (32-bit) : float in C
  - Double precision (64-bit) : double in C

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

IEEE Floating-Point Format



$$x = (-1)^{S} \times (1 + \text{Fraction}) \times 2^{(\text{Exponent} - \text{Bias})}$$

S : sign bit

- 0 ⇒ non-negative
- 1 ⇒ negative

Normalized significand : "*1.0 ≤ |significand| < 2.0*"

- Always has a leading pre-binary-point 1 bit, so no need to represent it explicitly (hidden bit)
- Significand is '**Fraction**' with the "1." implicitly restored

Exponent: excess representation : "*actual exponent + Bias*"

- or, "actual exponent = **Exponent** - Bias"
- Single: Bias = 127; Double: Bias = 1023
- Ensures exponent is unsigned

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

IEEE Floating-Point Format

Example

Represent -0.75 in IEEE floating point format.

A.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

IEEE Floating-Point Format

Example

Represent 0.625 in IEEE floating point format.

A.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

IEEE Floating-Point Format

Example

Represent -3.75 in IEEE floating point format.

A.

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Single-Precision Range

- **Exponents 00000000 and 11111111 are reserved**
  - exponent == 0, fraction == 0 ➔ **0**
  - exponent == 255, fraction == 0 ➔ **± ∞** (infinity)
  - exponent == 255, fraction != 0 ➔ **NaN**
    - Indicates illegal or undefined result (e.g., 0.0 / 0.0)
  - exponent == 0, fraction != ???

- **Smallest value (*)**
  - Exponent: 00000001 ⇒ actual exponent = 1 – 127 = –126
  - Fraction: 000…00 ⇒ significand = 1.0
  - $\pm 1.0 \times 2^{-126} \approx \pm 1.2 \times 10^{-38}$

- **Largest value**
  - Exponent: 11111110 ⇒ actual exponent = 254 – 127 = +127
  - Fraction: 111…11 ⇒ significand ≈ 2.0
  - $\pm 2.0 \times 2^{+127} \approx \pm 3.4 \times 10^{+38}$

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Double-Precision Range

- **Exponents 0000…00 and 1111…11 reserved**
  - Same idea as single-precision case for **0**, **± ∞** (infinity), and **NaN**.

- **Smallest value (*)**
  - Exponent: 00000000001 ⇒ actual exponent = 1 – 1023 = –1022
  - Fraction: 000…00 ⇒ significand = 1.0
  - $\pm 1.0 \times 2^{-1022} \approx \pm 2.2 \times 10^{-308}$

- **Largest value**
  - Exponent: 11111111110 ⇒ actual exponent = 2046 – 1023 = +1023
  - Fraction: 111…11 ⇒ significand ≈ 2.0
  - $\pm 2.0 \times 2^{+1023} \approx \pm 1.8 \times 10^{+308}$

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Floating-Point Precision

- **Relative precision**
  - all fraction bits are significant
  - Single: approx 2^{-23}
    - Equivalent to $23 \times \log_{10}2 \approx 23 \times 0.3 \approx 6$ decimal digits of precision
  - Double: approx 2^{-52}
    - Equivalent to $52 \times \log_{10}2 \approx 52 \times 0.3 \approx 16$ decimal digits of precision

- **Denormal Numbers**
  - Exponent = 000...0 ⇒ hidden bit is 0

$$x = (-1)^S \times (0 + \text{Fraction}) \times 2^{-\text{bias}}$$

  - Smaller than normalized numbers
    - allow for gradual underflow, with diminishing precision

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Floating-Point Addition

Consider a 4-digit 'decimal' example (for understanding)

- $9.999 \times 10^{1} + 1.610 \times 10^{-1}$

1. Align decimal points
   - Shift number with smaller exponent to match the exponent of the larger
   - $9.999 \times 10^{1} + 0.016 \times 10^{1}$
2. Add significands
   - $9.999 \times 10^{1} + 0.016 \times 10^{1} = 10.015 \times 10^{1}$
3. Normalize result & check for over/underflow
   - 1.0015×10^{2}
4. Round and renormalize if necessary
   - 1.002×10^{2}

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Floating-Point Addition

Now consider a 4-digit binary example : (0.5 + –0.4375)

- $1.000_2 \times 2^{-1} + -1.110_2 \times 2^{-2}$

1. Align binary points
   - Shift number with smaller exponent to match the exponent of the larger
   - $1.000_2 \times 2^{-1} + -0.111_2 \times 2^{-1}$
2. Add significands
   - $1.000_2 \times 2^{-1} + -0.111_2 \times 2^{-1} = 0.001_2 \times 2^{-1}$
3. Normalize result & check for over/underflow
   - $1.000_2 \times 2^{-4}$, with no over/underflow
4. Round and renormalize if necessary
   - $1.000_2 \times 2^{-4}$ (no change) $= 0.0625$

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Floating-Point Multiplication

Consider a 4-digit 'decimal' example (for understanding)

- $1.110 \times 10^{10} \times 9.200 \times 10^{-5}$

1. Add exponents
   - For biased exponents, subtract bias from sum
   - New exponent = 10 + –5 = 5
2. Multiply significands
   - $1.110 \times 9.200 = 10.212 \Rightarrow 10.212 \times 10^{5}$
3. Normalize result & check for over/underflow
   - 1.0212×10^{6}
4. Round and renormalize if necessary
   - 1.021×10^{6}
5. Determine sign of result from signs of operands
   - $+1.021 \times 10^{6}$

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

Floating-Point Multiplication

Now consider a 4-digit binary example of (0.5 × –0.4375)

- $1.000_2 \times 2^{-1} \times -1.110_2 \times 2^{-2}$

1. Add exponents
   - Unbiased: $-1 + -2 = -3$
   - Biased: $(-1 + 127) + (-2 + 127) - 127 = -3 + 254 - 127 = -3 + 127$
2. Multiply significands
   - $1.000_2 \times 1.110_2 = 1.1102 \Rightarrow 1.110_2 \times 2^{-3}$
3. Normalize result & check for over/underflow
   - $1.110_2 \times 2^{-3}$ (no change) with no over/underflow
4. Round and renormalize if necessary
   - $1.110_2 \times 2^{-3}$ (no change)
5. Determine sign: +ve × –ve ⇒ –ve
   - $-1.110_2 \times 2^{-3} = -0.21875$

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential

FP Instructions in MIPS

There is usually dedicated hardware for FP arithmetic

- Complex, and usually takes several cycles

FP arithmetic hardware usually does

- Addition, subtraction, multiplication, division, reciprocal, square-root
- FP ↔ integer conversion

Separate FP registers

- 32 single-precision: $f0, $f1, … $f31
- Paired for double-precision: $f0/$f1, $f2/$f3, …
  - Release 2 of MIPs ISA supports 32 × 64-bit FP reg's

FP instructions operate only on FP registers

- Programs generally don't do integer ops on FP data, or vice versa
- More registers with minimal code-size impact

Copyright © 2021 by CAU. All rights reserved. Strictly Confidential



Thank you

Bumsoo Kim

School of Computer Science & Engineering

E-mail : bumsoo@cau.ac.kr

Office : 310-430